# MLDL EXPERIMENT 9

## Aim:

Implement Recurrent Neural Network (RNN) / LSTM for time series or text data.

### Dataset Description:

The Reuters Newswire Topic Classification Dataset used in this experiment is a built-in dataset provided by the Keras deep learning library. It is a widely used benchmark dataset for multi-class text classification tasks in Natural Language Processing (NLP). The dataset contains preprocessed news articles labeled across 46 topic categories. The articles are already tokenized and converted into sequences of integer word indices based on word frequency, allowing direct usage in deep learning models without manual text cleaning or tokenization. Since each article is paired with a topic label (0 to 45), it is a supervised learning dataset suitable for training Recurrent Neural Networks (RNNs) and LSTM models. Due to its multi-class structure, standardized preprocessing, and moderate dataset size, the Reuters dataset is commonly used in academic research, deep learning tutorials, and multi-class classification experiments.

- **File Type:** Built-in Keras dataset (NumPy arrays loaded via TensorFlow/Keras)
- **Dataset Size:** 11,228 newswire articles (8,982 training samples & 2,246 testing samples)
- **Data Format:** Integer-encoded sequences representing word indices
- **Number of Classes:** 46 topic categories (0 to 45)

| Column Name | Data Type | Description |
|---|---|---|
| Article (Sequence) | Numerical (Integer sequence) | Tokenized news article represented as a sequence of word indices based on frequency ranking. |
| Label | Numerical (Integer: 0-45) | Topic category corresponding to each article (0 to 45 representing 46 different news topics). |

**Dataset Source:** https://keras.io/api/datasets/reuters/

## Recurrent Neural Network (RNN) / LSTM

### Theory:

Recurrent Neural Networks (RNNs) are a class of deep learning models designed for processing sequential data such as text, speech, and time-series signals. Unlike traditional feedforward neural networks, RNNs maintain a hidden state that captures information from previous time steps, allowing them to model temporal dependencies. However, standard RNNs suffer from the vanishing gradient problem, which makes learning long-term dependencies difficult. To overcome this

limitation, Long Short-Term Memory (LSTM) networks were introduced. LSTMs are a specialized type of RNN that use gating mechanisms to control the flow of information, enabling them to retain relevant information over long sequences.

The input to an RNN is a sequence matrix:

$$\mathbf{X} \in \mathbb{R}^{(T \times d)}$$

where:

- T = length of the sequence (number of time steps / words)
- d = dimension of each input vector (e.g., embedding dimension)

At each time step t, the RNN updates its hidden state as:

$$h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{(t-1)} + b)$$

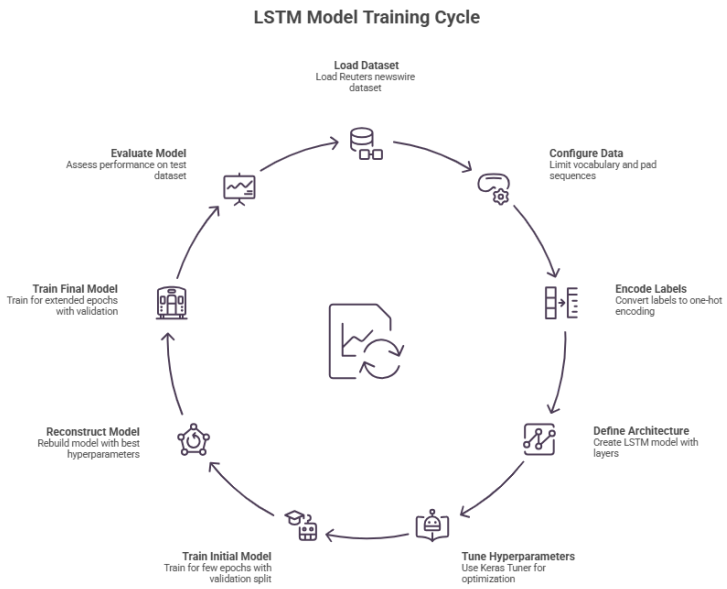
where:

- $x_t$ = input at time step t
- $h_t$ = hidden state at time step t
- $W_x$ = input weight matrix
- $W_h$ = recurrent weight matrix
- b = bias term

## Limitations:

1. **Vanishing and Exploding Gradient Problem:** Traditional Recurrent Neural Networks suffer from the vanishing and exploding gradient problem during backpropagation through time (BPTT). When sequences are long, gradients can shrink exponentially (vanish) or grow uncontrollably (explode), making training unstable and preventing the model from learning long-term dependencies effectively. Although LSTMs were specifically designed to mitigate the vanishing gradient issue using gating mechanisms, they are not completely immune, especially for extremely long sequences.
2. **Difficulty in Modeling Very Long-Term Dependencies:** While LSTMs improve upon standard RNNs by introducing memory cells and gates, they still struggle when dependencies span very long sequences. In practical NLP tasks with lengthy documents, important contextual information may still be diluted over time. More advanced architectures such as Transformers often outperform RNN/LSTM models in capturing long-range relationships.
3. **High Computational Time Due to Sequential Processing:** RNNs and LSTMs process data sequentially, meaning each time step depends on the previous one. This limits parallelization during training and increases computation time compared to feedforward networks or Transformer-based models. For large datasets and long sequences, training becomes slower and less efficient, even with GPU acceleration.
4. **Large Number of Parameters in LSTM:** LSTMs contain multiple gates (input, forget, output) with separate weight matrices, significantly increasing the number of parameters compared to simple RNNs. This increases memory usage and computational cost. In the Reuters multi-class setting (46 categories), the output layer further amplifies complexity. Larger models also increase the risk of overfitting when the training dataset is not sufficiently large.

## Workflow:




1. **Dataset Loading:** The built-in Keras Reuters newswire dataset is loaded using TensorFlow. It contains 8,982 training articles and 2,246 testing articles labeled across 46 topic categories. Each article is preprocessed and represented as a sequence of integer word indices based on word frequency ranking.
2. **Data Configuration and Vocabulary Limitation:** Only the top 10,000 most frequent words are considered (MAX_FEATURES = 10000). This reduces vocabulary size, lowers computational cost, and removes rare words that contribute little to model learning.
3. **Sequence Padding:** Since articles have variable lengths, all sequences are padded (or truncated) to a fixed length of 200 words (MAX_LEN = 200) using pad_sequences(). This ensures uniform input shape required for batch processing in the LSTM model.
4. **Label Encoding:** Labels are one-hot encoded using to_categorical() to convert integer class labels (0–45) into 46-dimensional binary vectors required for multi-class softmax output.
5. **Model Architecture Definition (Embedding + LSTM):** A Sequential deep learning model is defined consisting of Embedding Layer, LSTM Layer, Dropout Layer, and Dense Output Layer with softmax activation for 46-class classification.
6. **Hyperparameter Tuning (Keras Tuner – Random Search):** The RandomSearch tuner explores different hyperparameter combinations. The objective is to maximize validation accuracy. Multiple trials (5 trials) are executed to find the best-performing configuration.
7. **Initial Search Training Phase:** Each hyperparameter configuration is trained for a small number of epochs (3 epochs) with a validation split of 20% to quickly evaluate performance and identify optimal parameters.
8. **Best Model Reconstruction:** After tuning, the best hyperparameters are selected, and the model is rebuilt using those optimal values to ensure improved generalization performance.
9. **Final Model Training:** The optimized model is trained for extended epochs (7 epochs) using 80% of the training data while reserving 20% for validation. This phase allows the model to fully learn topic classification patterns from text sequences.
10. **Model Evaluation:** The trained LSTM model is evaluated on the unseen test dataset to compute final test accuracy and loss. This measures the model's ability to generalize to new articles.

## Hyperparameter Tuning:

LSTM-based Recurrent Neural Networks are highly sensitive to hyperparameter choices, as they directly influence memory capacity, learning stability, convergence speed, and generalization performance. Manual tuning is inefficient due to the large combination space of embedding size, hidden units, dropout, and learning rate. Therefore, automated hyperparameter tuning is performed using Keras Tuner with the Random Search algorithm. The tuner evaluates multiple LSTM configurations and selects the best-performing model based on validation accuracy (Best val_accuracy achieved during search: 56.43%). The tuned hyperparameters in this experiment are:

1. **Embedding Dimension:** The embedding layer converts integer word indices into dense vector representations. The embedding dimension is tuned between 32–128 in steps of 32. The optimal configuration selected an embedding dimension of 64, providing a moderate-sized word representation suitable for capturing semantic relationships across 46 news topics.
2. **Number of LSTM Units:** The LSTM layer's hidden units determine the model's ability to capture sequential dependencies and contextual relationships within text. The search range is 32–128 units in steps of 32. The best configuration selected 64 LSTM units, providing a balanced trade-off between learning expressive sequence representations and maintaining generalization on the multi-class Reuters task.
3. **Dropout Regularization Rate:** Dropout is tuned between 0.2–0.5 in steps of 0.1 to reduce overfitting by randomly disabling neurons during training. The tuner selected a 0.4 dropout rate, indicating stronger regularization is needed due to the high complexity of 46-class classification.
4. **Learning Rate:** Tuned among 0.01, 0.001, and 0.0001. The optimal value selected was 0.001, ensuring stable and smooth convergence during training.

| Hyperparameter | Optimal Value | Search Range |
|---|---|---|
| Embedding Dimension | 64 | 32 to 128 (step 32) |
| LSTM Units | 64 | 32 to 128 (step 32) |
| Dropout Rate | 0.4 | 0.2 to 0.5 (step 0.1) |
| Learning Rate | 0.001 | 0.01, 0.001, 0.0001 |

## Performance Analysis:

The performance of the hyperparameter-tuned LSTM-based RNN is evaluated using validation accuracy during tuning, training vs validation learning curves, and final test set performance. Since the Reuters dataset is a supervised multi-class text classification problem with 46 categories, evaluation focuses on topic prediction accuracy and generalization capability.

**Test Set Performance:**

a. Test Accuracy: 58.06%
b. Test Loss: 1.6460

The test accuracy of 58.06% reflects the inherent difficulty of 46-class classification with a single-layer LSTM. The model significantly outperforms a random baseline (which would achieve ~2.17% for 46 classes), demonstrating meaningful learning of textual patterns. The high loss value (1.6460) is expected given the large number of output classes.

**Generalization Analysis:**

c. Training Accuracy: ~59% (by epoch 7)
d. Validation Accuracy: ~55–58%
e. Test Accuracy: 58.06%

The training and validation accuracy curves show a gradual, steady improvement across all 7 epochs, indicating the model is actively learning without severe overfitting. Training loss consistently decreased from ~2.5 to ~1.4, while validation loss also declined steadily, confirming healthy convergence.

**Confusion Matrix Interpretation:**

The confusion matrix for the top 10 most frequent classes reveals clear patterns in model performance. Class 3 (earnings/finance) and Class 4 (trade) achieved the highest correct classifications with 736 and 404 correct predictions respectively, reflecting their large representation in the dataset. Classes with fewer samples (such as 8, 11, 13, 16, 20, 25) showed weaker performance, with the model frequently misclassifying them as Class 1 or Class 19 — the two dominant classes. This is characteristic of class imbalance in the Reuters dataset.

**Prediction Probability Distribution:**

The prediction probability distribution shows a bimodal pattern — a large cluster near 0.0–0.2 (uncertain predictions across many minority classes) and a dominant spike near 0.95–1.0 (high-confidence predictions mostly for Classes 3 and 4). This confirms the model has learned to be highly confident for dominant classes while remaining uncertain for underrepresented topics, which is a known behaviour in class-imbalanced multi-class datasets.

**Classification Report Summary (Key Classes):**

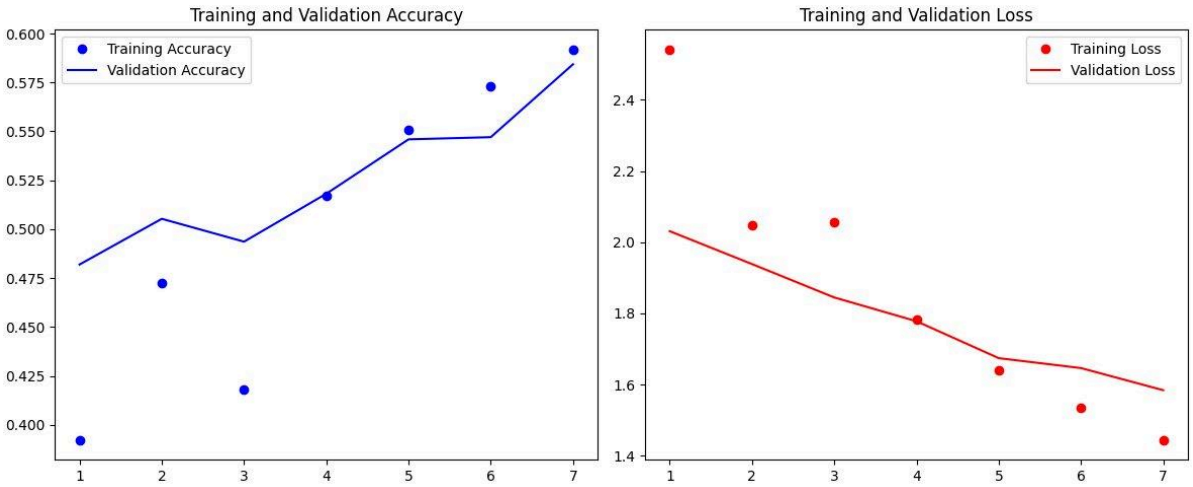
| Class | Topic | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|---|
| 3 | Earnings/Finance | 0.92 | 0.91 | 0.91 | 813 |
| 4 | Trade | 0.76 | 0.85 | 0.80 | 474 |
| 19 | Money/FX | 0.29 | 0.65 | 0.40 | 133 |
| 1 | Cocoa | 0.13 | 0.44 | 0.21 | 105 |
| 16 | Crude Oil | 0.14 | 0.15 | 0.14 | 99 |
| **Overall (Weighted)** | - | **0.53** | **0.58** | **0.55** | **2246** |

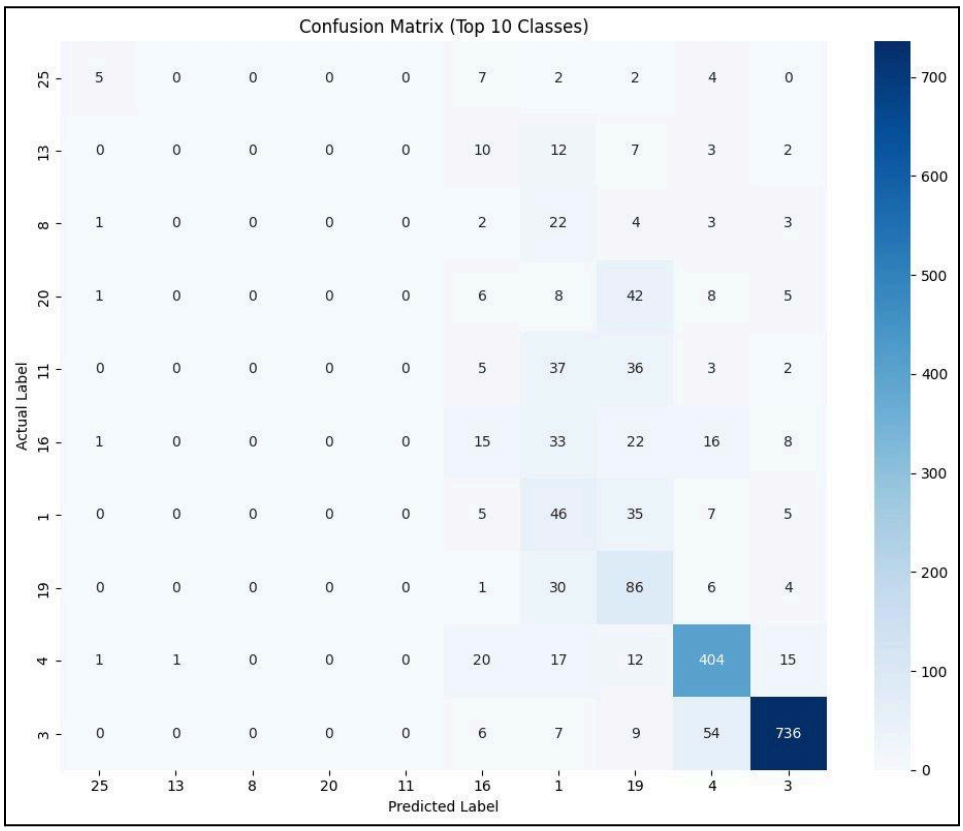
## Code:

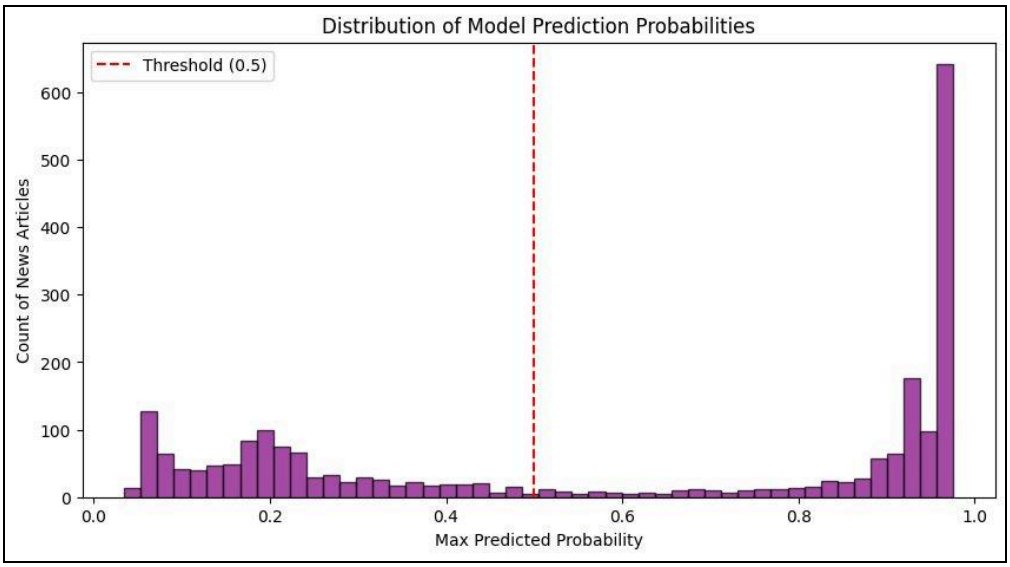
```
!pip install keras-tuner -q

# Step 2: Imports
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import reuters
from tensorflow.keras.preprocessing.sequence import
pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM,
Dense, Dropout
import keras_tuner as kt
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix,
classification_report

# Step 3: Configuration
MAX_FEATURES = 10000   # Top 10,000 words
MAX_LEN = 200          # Pad/truncate to 200 words
NUM_CLASSES = 46       # Reuters has 46 topic
categories

# Step 4: Load Dataset
print("Loading Reuters dataset...")
(x_train, y_train), (x_test, y_test) =
reuters.load_data(num_words=MAX_FEATURES)
print(f"Training sequences : {len(x_train)}")
print(f"Testing sequences  : {len(x_test)}")
print(f"Number of classes  : {NUM_CLASSES}")

# Step 5: Pad Sequences
x_train = pad_sequences(x_train, maxlen=MAX_LEN)
x_test  = pad_sequences(x_test,  maxlen=MAX_LEN)
print(f"Shape after padding - Train: {x_train.shape}, Test:
{x_test.shape}")

# Step 6: Convert labels to categorical
y_train_cat = tf.keras.utils.to_categorical(y_train,
NUM_CLASSES)
y_test_cat  = tf.keras.utils.to_categorical(y_test,
NUM_CLASSES)

# Step 7: Model builder for Keras Tuner
def build_model(hp):
    model = Sequential()
    model.add(Embedding(
        input_dim=MAX_FEATURES,
        output_dim=hp.Int('embedding_dim',
min_value=32, max_value=128, step=32),
        input_length=MAX_LEN
    ))
    model.add(LSTM(
        units=hp.Int('lstm_units', min_value=32,
max_value=128, step=32)
    ))
    model.add(Dropout(
        rate=hp.Float('dropout_rate', min_value=0.2,
max_value=0.5, step=0.1)
    ))
    model.add(Dense(NUM_CLASSES,
activation='softmax'))  # Multi-class output

    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=hp.Choice('learning_rate',
values=[1e-2, 1e-3, 1e-4])
        ),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

# Step 8: Hyperparameter Tuning with Random Search
tuner = kt.RandomSearch(
    build_model,
    objective='val_accuracy',
    max_trials=5,
    executions_per_trial=1,
    directory='reuters_tuning_dir',
    project_name='reuters_lstm_tuning'
)

tuner.search_space_summary()

print("\nStarting Hyperparameter Search...")
tuner.search(
    x_train, y_train_cat,
    epochs=3,
    validation_split=0.2,
    batch_size=64
)

# Step 9: Best Hyperparameters
best_hps =
tuner.get_best_hyperparameters(num_trials=1)[0]
print(f"""
Hyperparameter Search Complete!
  Best Embedding Dim  :
{best_hps.get('embedding_dim')}
  Best LSTM Units     : {best_hps.get('lstm_units')}
  Best Dropout Rate   : {best_hps.get('dropout_rate')}
  Best Learning Rate  : {best_hps.get('learning_rate')}
""")

# Step 10: Final Training with Best Hyperparameters
model = tuner.hypermodel.build(best_hps)
history = model.fit(
    x_train, y_train_cat,
    epochs=7,
    validation_split=0.2,
    batch_size=64
)

# Step 11: Training & Validation Curves
```

```
acc     = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss    = history.history['loss']
val_loss= history.history['val_loss']
epochs_range = range(1, len(acc) + 1)

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc,     'bo', label='Training
Accuracy')
plt.plot(epochs_range, val_acc, 'b',  label='Validation
Accuracy')
plt.title('Training and Validation Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss,     'ro', label='Training Loss')
plt.plot(epochs_range, val_loss, 'r',  label='Validation
Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.tight_layout()
plt.show()

# Step 12: Test Evaluation
print("\nEvaluating on Test Data...")
test_loss, test_acc = model.evaluate(x_test, y_test_cat)
print(f"Test Accuracy : {test_acc * 100:.2f}%")
print(f"Test Loss     : {test_loss:.4f}")

# Step 13: Predictions
y_pred_probs = model.predict(x_test)
y_pred = np.argmax(y_pred_probs, axis=1)

# Step 14: Classification Report
print("\nClassification Report (Top classes):")
print(classification_report(y_test, y_pred))

# Step 15: Confusion Matrix (Top 10 most frequent
classes only — 46 is too many to display)
top_classes = np.argsort(np.bincount(y_test))[-10:]
mask = np.isin(y_test, top_classes)
cm = confusion_matrix(y_test[mask], y_pred[mask],
labels=top_classes)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=top_classes,
yticklabels=top_classes)
plt.title('Confusion Matrix (Top 10 Classes)')
plt.xlabel('Predicted Label')
plt.ylabel('Actual Label')
plt.tight_layout()
plt.show()

# Step 16: Prediction Probability Distribution
plt.figure(figsize=(10, 5))
max_probs = np.max(y_pred_probs, axis=1)
plt.hist(max_probs, bins=50, alpha=0.7, color='purple',
edgecolor='black')
plt.title('Distribution of Model Prediction Probabilities')
plt.xlabel('Max Predicted Probability')
plt.ylabel('Count of News Articles')
plt.axvline(x=0.5, color='red', linestyle='--',
label='Threshold (0.5)')
plt.legend()
plt.show()

# Step 17: Sample Predictions
word_index = reuters.get_word_index()
index_word = {v: k for k, v in word_index.items()}

print("\nSample Predictions:")
for i in range(3):
    review_words = ' '.join([index_word.get(idx - 3, '?') for
idx in x_test[i] if idx > 3])
    print(f"\nArticle snippet : ...{review_words[-200:]}")
    print(f"Actual class     : {y_test[i]}")
    print(f"Predicted class : {y_pred[i]}")
    print(f"Confidence       : {max_probs[i]:.4f}")
```

## Output:


Training and Validation Accuracy
Training Accuracy
Validation Accuracy
0.600
0.575
0.550
0.525
0.500
0.475
0.450
0.425
0.400
1 2 3 4 5 6 7
Training and Validation Loss
Training Loss
Validation Loss
2.4
2.2
2.0
1.8
1.6
1.4
1 2 3 4 5 6 7


Confusion Matrix (Top 10 Classes)
Actual Label
Predicted Label
25 13 8 20 11 16 1 19 4 3
25: 5 0 0 0 0 7 2 2 4 0
13: 0 0 0 0 0 10 12 7 3 2
8: 1 0 0 0 0 2 22 4 3 3
20: 1 0 0 0 0 6 8 42 8 5
11: 0 0 0 0 0 5 37 36 3 2
16: 1 0 0 0 0 15 33 22 16 8
1: 0 0 0 0 0 5 46 35 7 5
19: 0 0 0 0 0 1 30 86 6 4
4: 1 1 0 0 0 20 17 12 404 15
3: 0 0 0 0 0 6 7 9 54 736
700
600
500
400
300
200
100
0




## Conclusion:

The experiment successfully implemented and tuned an LSTM-based Recurrent Neural Network for multi-class topic classification using the Reuters Newswire dataset. The hyperparameter-optimized model achieved a test accuracy of 58.06% across 46 topic categories, significantly outperforming the random baseline of ~2.17%. The best hyperparameter configuration identified embedding dimension of 64, 64 LSTM units, dropout rate of 0.4, and learning rate of 0.001. The training and validation curves demonstrated steady convergence across 7 epochs without signs of severe overfitting. The confusion matrix revealed that dominant classes (3 - Earnings and 4 - Trade) were classified with high accuracy (F1-scores of 0.91 and 0.80 respectively), while minority classes suffered from class imbalance effects. The prediction probability distribution confirmed confident predictions for dominant classes and uncertainty for underrepresented ones. Overall, the experiment confirms that LSTM networks are capable of multi-class text classification, while also highlighting that class imbalance handling, deeper architectures, or Transformer-based models could further improve performance on the Reuters dataset.